

Multi-Source Candidate Data Transformer — Technical Design

Pranati Dash · pranatidash727@gmail.com · Eightfold Engineering Intern Assignment (Jul–Dec 2026)

Pipeline

detect → **extract** → **normalize** → **merge** → **score_confidence** → **project** → **validate**. Detect identifies source type from file extension/shape (csv, ats.json, resume.pdf/.docx, notes.txt). Extract maps each source's native fields into a common, loosely-typed *RawRecord* (one per source, per candidate) without dropping unrecognized data. Normalize runs per-field cleaners (dates, phone, country, skills) independently of source so the same logic applies everywhere. Merge groups *RawRecords* into one *CanonicalProfile* per person and resolves conflicts. Score_confidence assigns per-field confidence using source agreement and source reliability. Project applies the runtime config to produce the requested output shape. Validate checks the projected JSON against a schema before returning it.

Canonical Schema & Normalization

Uses the default schema as given. Normalizations: dates → YYYY-MM (partial dates allowed, day dropped; unparseable → null, never guessed); phones → E.164 using a default-region heuristic (country from location field, else assume US, else null if ambiguous); country → ISO-3166 alpha-2 via a small lookup table with common-name aliases; skills → canonicalized via a lowercase + alias map (e.g. "JS"/"Javascript" → "javascript"), unknown skills pass through as-is (never dropped, never invented).

Merge & Conflict Resolution

Match key: normalized primary email if present on ≥ 2 sources; otherwise normalized (full_name + current_company) as a fallback fuzzy key. Records that don't match anything become their own candidate_id.

Field-level winner rule (source priority, not source order): structured sources (CSV, ATS) outrank unstructured (resume, notes, scraped profiles) for factual fields like phone/title/company, since structured data is operator-entered and less error-prone to parse. Within the same tier, most-recently-seen wins. List fields (emails, phones, skills) are **unioned** across sources, deduplicated, not overwritten — losing a valid second email is worse than keeping both.

Confidence: field confirmed by 2+ independent sources with agreeing values → 0.9; single structured source → 0.7; single unstructured/parsed-from-prose source → 0.4–0.5; conflicting values across sources → winner kept but confidence capped at 0.5 to flag it for review. overall_confidence = weighted mean across populated fields.

Runtime Config (Projection Layer)

The *CanonicalProfile* is never mutated for output. A separate *Projector* reads the config's field list (path, from, type, normalize, required) and walks the canonical object using dot/bracket paths to pull each value, applying any field-specific normalize override. include_confidence/include_provenance toggles add or strip those sub-objects per field. on_missing is applied per field after extraction: null inserts None, omit drops the key entirely, error raises a validation failure listing every offending required field (not just the first). Output is then checked against a JSON-schema generated from the config itself before being returned, so a bad config can't silently produce malformed JSON.

Edge Cases Handled (and what's descoped)

- Missing/empty/corrupt source file → that extractor returns zero *RawRecords* and logs a warning; pipeline continues with remaining sources rather than crashing.
- Conflicting values across sources (two different phones) → both kept where the field is list-typed; scalar fields apply the priority rule above and confidence is capped, never silently picked with high confidence.
- Same person split across sources with different emails → caught by the name+company fallback key; if it still doesn't merge, candidate is descoped (left as separate profiles) rather than risking an incorrect merge.
- Unparseable/partial dates in resumes ("Summer 2022", "present") → best-effort YYYY-MM extraction; if not confidently parseable, field is null with provenance noting the raw source text, never a guessed date.
- Skill spelled/cased differently across sources ("Python", "python3") → canonicalized via alias map; sources[] on the skill entry still lists every source that mentioned it.

Deliberately left out under time pressure: true fuzzy/ML-based name matching for dedup (only exact-email and name+company fallback are implemented); LinkedIn/GitHub live scraping (config supports it structurally but only CSV/ATS + resume/notes extractors are built for the sample run); a UI beyond a CLI, since the assignment marks this lower priority.